

ML Observability (Concepts, Definitions & Foundations)

1 Motivation

Machine Learning (ML) systems differ from traditional software systems because their behavior depends heavily on data distributions. When deployed in real-world environments, these distributions can change over time, causing model degradation. This creates the need for **ML Observability**, which ensures we can detect, understand, and debug such changes.

2 What is ML Observability?

Definition: ML Observability is the ability to **measure, monitor, and explain** the internal behavior of an ML system using external signals such as logs, metrics, and statistical properties of data. Idea is not just detecting failure, but understanding *why* it happened.

3 Observability vs Monitoring

Monitoring focuses on predefined signals (e.g., accuracy drops below threshold). Observability allows answering unknown questions by analyzing system outputs.

Example:

- Monitoring: “Accuracy dropped from 92% to 80%”
- Observability: “Accuracy dropped because input feature distribution shifted due to seasonal change”

4 Core Components of Observability

4.1 Metrics

Definition: Numerical values measured over time that summarize system behavior.

Examples:

- Model accuracy: proportion of correct predictions
- Latency: time taken for prediction
- Feature mean/variance

4.2 Logs

Definition: Detailed records of events that occur during system execution.

Examples:

- Input features used for prediction
- Output predictions
- Error messages

4.3 Traces

Definition: End-to-end tracking of a single request through different components of a system.

Example:

- User request $\rightarrow$ API $\rightarrow$ Feature pipeline $\rightarrow$ Model inference

5 ML-Specific Observability Concepts

5.1 Data Distribution

Definition: The statistical properties of data, including mean, variance, and probability distribution.

Example:

- Training data age distribution: mostly 20–40
- Production data age distribution: mostly 50–70

5.2 Data Drift

Definition: Data drift occurs when the distribution of input features changes over time.

$$P_{train}(X) \neq P_{production}(X) \tag{1}$$

Example:

- A fraud model trained on weekday transactions now sees weekend-heavy traffic

5.3 Types of Data Drift

5.3.1 Covariate Shift

Definition: The distribution of input variables changes, but the relationship between input and output remains the same.

$$P(X)_{train} \neq P(X)_{production}, \quad P(Y|X) \text{ unchanged} \tag{2}$$

Intuition: Same rule, different data.

Example:

- A model trained on daytime traffic now receives nighttime traffic data

5.3.2 Prior Probability Shift

Definition: The distribution of the target variable changes.

$$P(Y)_{train} \neq P(Y)_{production} \quad (3)$$

Example:

- Fraud rate increases from 2% to 10%

5.4 Concept Drift

Definition: The relationship between input features and output changes over time.

$$P(Y|X)_{train} \neq P(Y|X)_{production} \quad (4)$$

Intuition: The underlying rules have changed.

Example:

- Customer behavior changes after a new policy or pandemic

5.5 Prediction Drift

Definition: The distribution of model outputs changes over time.

Example:

- Model starts predicting more positive cases than before

5.6 Data Quality Issues

Definition: Problems in data that degrade model performance.

Examples:

- Missing values: null or NaN entries
- Schema mismatch: feature names/types change
- Outliers: extreme values outside expected range

6 Statistical Methods for Observability

6.1 Kullback-Leibler (KL) Divergence

Definition: Measures how one probability distribution differs from another. Kullback-Leibler (KL) Divergence measures how much one probability distribution diverges from another by quantifying the “information loss” when one distribution is used to approximate the other. In practical terms, it answers the question: *how incorrect would our assumptions be if we believed that production data still followed the training distribution?*

The key idea behind KL divergence is asymmetry. It treats one distribution as the reference (typically training data) and measures how surprising the observed production data is under that assumption. If production data contains patterns that were rare or unseen during training, KL divergence increases significantly, signaling potential drift.

Consider a fraud detection model trained on transaction amounts mostly between \$10 and \$200. If, in production, the system begins receiving a large number of transactions above \$1000, KL divergence will increase sharply. This happens because the model was not exposed to such values during training, making these observations highly “unexpected” under the learned distribution.

In MLOps pipelines, KL divergence is particularly useful when detecting subtle but important shifts, especially in cases where rare events carry high importance.

$$D_{KL}(P||Q) = \sum P(x) \log \frac{P(x)}{Q(x)} \quad (5)$$

Interpretation:

- 0 means identical distributions
- Higher values indicate greater divergence

6.2 Jensen-Shannon Divergence

Definition: A symmetric version of KL divergence. Jensen-Shannon (JS) Divergence is a refined and more stable alternative to KL divergence. Instead of directly comparing the training distribution to the production distribution, it introduces an intermediate step by computing a midpoint (average distribution) and measuring how both distributions deviate from this shared reference. This modification makes JS divergence symmetric, meaning the distance from training to production is the same as from production to training. More importantly, it avoids the instability issues of KL divergence, particularly when dealing with very small probabilities or uneven distributions.

In a real-world MLOps setting, consider monitoring user behavior features such as session duration or click frequency. These features often contain noise and irregular patterns, especially in large-scale systems. JS divergence provides a more stable signal in such cases, preventing false alarms that might arise due to outliers or rare events.

Because of its stability and interpretability, JS divergence is often preferred in general-purpose drift detection systems, especially when integrated into automated monitoring tools.

6.3 Population Stability Index (PSI)

Definition: Measures how much a variable’s distribution has changed. Population Stability Index (PSI) takes a more practical and industry-oriented approach compared to KL and JS divergence. Instead of working directly with continuous probability distributions, PSI operates on binned data, making it particularly suitable for tabular datasets commonly used in business applications. The process begins by dividing a feature into predefined ranges (bins), such as income brackets or age groups. The proportion of data falling into each bin is then calculated separately for training and production datasets. PSI measures how much these proportions differ across the bins, providing a clear indication of distribution shift.

For example, consider a credit scoring model that uses customer income as a feature. During training, suppose 60% of customers fall within the \$30k–\$60k range. In production, if this proportion drops to 30%, PSI will capture this shift explicitly. Unlike KL or JS divergence, which require interpretation of abstract values, PSI provides well-established thresholds that guide decision-making. In industry practice, a PSI value below 0.1 indicates negligible change, values between 0.1 and 0.25 suggest moderate drift, and values above 0.25 typically trigger investigation or retraining. This threshold-based interpretation makes PSI highly actionable and one of the most widely used techniques in production MLOps systems.

$$PSI = \sum (Actual - Expected) \cdot \log \left(\frac{Actual}{Expected} \right) \quad (6)$$

Interpretation:

- < 0.1 : no significant change
- $0.1 - 0.25$: moderate shift
- > 0.25 : significant drift

7 Uncertainty in ML

7.1 Aleatoric Uncertainty

Definition: Uncertainty due to inherent randomness in data.

Example:

- Noisy sensor readings

7.2 Epistemic Uncertainty

Definition: Uncertainty due to lack of knowledge (limited training data).

Example:

- Model encountering unseen scenarios

8 Key Observability Tasks

8.1 Data Monitoring

Tracking:

- Distribution changes
- Missing values

8.2 Model Monitoring

Tracking:

- Prediction distributions
- Confidence scores

8.3 Pipeline Monitoring

Tracking:

- Failures in preprocessing
- Feature engineering issues

9 Observability Workflow

1. Collect data (inputs, outputs, metadata)
2. Store logs and metrics
3. Analyze distributions
4. Detect anomalies
5. Trigger alerts
6. Retrain or rollback model

10 Common Mistakes

- Monitoring only accuracy
- Ignoring data drift
- No feature-level analysis
- No retraining pipeline

11 Summary

ML Observability extends traditional monitoring by incorporating:

- Statistical analysis
- Data distribution tracking
- Model behavior understanding

It is essential for maintaining reliable ML systems in production.

12 References

- Sculley et al., “Hidden Technical Debt in Machine Learning Systems,” NeurIPS, 2015.
- Gama et al., “A Survey on Concept Drift Adaptation,” ACM Computing Surveys, 2014.
- Breck et al., “The ML Test Score: A Rubric for ML Production Readiness,” Google Research, 2017.
- Amershi et al., “Software Engineering for Machine Learning,” ICSE, 2019.
- Google Cloud, “MLOps: Continuous Delivery and Automation Pipelines,”
<https://cloud.google.com/architecture/mlops-continuous-delivery-and-automation-pipelines>
- Evidently AI Documentation,
<https://docs.evidentlyai.com>

- Prometheus Documentation,
<https://prometheus.io/docs>
- OpenTelemetry Documentation,
<https://opentelemetry.io/docs>